

ANALYSIS REPORT FOR HMLR NLP DOCUMENT CLASSIFIER

HMLR NLP Document Classifier Implemented using Text Extraction and Entity Extraction

Exploratory Data Analysis (EDA)

The PDF had four scanned pages of anonymised planning decision notices. Since there was no digital text, I used Tesseract OCR with a zoom factor of 5 on all pages. The amount of text ranged from 1,079 characters on Page 4 to 3,575 on Page 3. Average word lengths ranged from 4.8 to 5.1 characters, suggesting the OCR results were generally good.

Keyword analysis showed that each page had a different text profile. Page 1 included phrases like "conditions imposed" and consent entries. Although the OCR did not find the exact phrase "planning charges", the zero-shot classifier still correctly identified the page type based on the overall text, demonstrating that transformer-based classification can be better than keyword matching alone. Pages 2 and 4 both used the "notice of approval" and "particulars of application/decision" format; Page 4 also used "approval of details" language. Page 3 featured "grant of conditional planning permission" and information about appeal rights from North Devon District Council, with a note that applicant details were on the back. This analysis helped define four classification categories.

Application numbers were mainly in two formats: Letter/YY/NNNN (like P/00/0759) and NN/YY/NNNN (like 02/80/1609). Since dates in the DD/MM/YYYY format could appear as NN/YY/NNNN, I added a date-filtering step. Applicant names usually came after keywords such as "Applicant:" and "approval granted to". However, the OCR often misread "Applicant" as "npl icant" or "Applivcarnt", so I used a regex-based backup extraction method along with NER.

Approach

The pipeline works in three main stages, each building on the last. For text extraction, it uses PyMuPDF to pull text directly, and switches to Tesseract OCR if needed. An automated quality check decides which method to use for each page. For page classification, it applies zero-shot classification with the facebook/bart-large-mnli model from Hugging Face, sorting pages into four EDA-based categories without needing training data. For entity extraction, it uses regex patterns and date filters to find application numbers and filter out dates. It also uses spaCy's transformer-based NER (en_core_web_trf) to identify applicant names, with regex backup patterns that look for text after certain keywords. Contextual filtering helps tell applicant names apart from officers and signatories.

Classification Result:

Page	Classification	Confidence	Application Numbers	Applicant Names
1	Planning Charges Register	0.3521	02/80/1609; 02/81/1237	Mr. & Mrs. J.M Doe Connell; My First Company Ltd.; Mr. & Mrs.J.M Doe
2	Decision Notice for Planning Permission	0.5719	P/00/0759; JK/2000/FS/1	Mr M Dale
3	Decision Notice for Conditional Planning Permission	0.4223	None found	None identified
4	Decision Notice for Approval of Details	0.4774	P/98/0964; P/96/0900	Mrs AM Stephen

Interpretation - Page classification was accurate for all four pages. Application numbers were extracted correctly wherever they appeared. Page 3 returned no results, which is expected since the applicant details and reference number are on the back of the document. Name extraction worked for applicants on Pages 1, 2, and 4. On Page 1, OCR errors included capturing 'Connell' as part of the applicant's name and listing the same applicant twice because of differences in the title ('Mrs' instead of 'Mrs'). On Page 4, the applicant's name was only partially captured as 'Mrs AM Stephen' instead of 'Mrs AM Stephens' because the last character was lost due to OCR quality.

Limitations

The main challenge was the quality of the OCR. The scanned documents varied in age and condition, which led to inconsistent text. For example, Page 3 was scanned too dark, and Page 1 had noisy form headers. The OCR also misread the word "Applicant" on Pages 2 and 4, and sometimes changed applicant names, like writing "Mrs" instead of "Mrs" or "Mrs" instead of "Mrs". These issues made it harder for spaCy to recognize entities because it needs clear text. Zero-shot classification confidence scores ranged from 0.35 to 0.57. Although all classifications were correct, these moderate scores show some uncertainty when using zero-shot inference on OCR text without training data for this task. Since there were only four pages, it is hard to validate the approach statistically or to find other types of pages that might appear in larger sets. Sometimes, name and location words were combined in the OCR text if there was no clear separator, so location words ended up included with applicant names.

Alternative methods

Improving image quality before running OCR helps extract text more accurately, especially from poor scans like Page 3. You can use tools like OpenCV for steps such as binarisation, deskewing, and noise reduction. If HMLR collects more labelled data, training a supervised classifier like a fine-tuned BERT or RoBERTa could give more reliable results and higher confidence scores than zero-shot inference. This would be a sensible next step. Fine-tuning a spaCy or Hugging Face NER model on planning document text can help it better recognise applicant names, application numbers, and other details, even if OCR makes mistakes. Using a CNN or Vision Transformer to classify pages by their visual layout, not just the text, can also help with OCR quality issues. This approach could work together with NLP in a multi-modal pipeline, combining visual classification with text-based entity recognition.